

ALU Logo Blockchain Protection Project Report

Student Submission Package



Abstract

This project protects the ALU official logo by registering its exact file fingerprint on an Ethereum blockchain and by tokenising ownership of that logo into fungible shares. The solution combines an ERC-721 NFT for uniqueness and an ERC-20 token for shared ownership.

Introduction

ALU needs a reliable technical way to prove which logo file is authentic. By storing the SHA-256 hash of the real logo file on-chain, the project creates a permanent verification record that is harder to alter than a private database and easier to compare than a traditional document-only copyright workflow.

Blockchain Setup

The environment uses Node.js, Hardhat, and OpenZeppelin. Hardhat compiles Solidity, runs a local Ethereum-compatible network, deploys contracts, and executes tests. A local blockchain is useful because it is free, resettable, and fast. A wallet address is the public identifier used to own tokens and submit transactions on the chain.

Smart Contract Explanation

The ALUAssetRegistry contract extends ERC-721 and stores an AssetMetadata struct containing the asset name, file type, content hash, registering address, and registration timestamp. registerAsset() blocks duplicate hashes, mints the NFT, stores metadata, and emits AssetRegistered. verifyLogoIntegrity() compares a supplied hash to the stored hash and returns both a boolean and a plain-language message. getAsset() returns the full metadata for a token.

Real ALU Logo Hash

The included official ALU logo file, downloaded from ALU's public website, produced the SHA-256 hash 727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560. The bytes32 value used in the contracts is 0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560. Even a tiny visual change would produce a different hash, which is why hashing is effective for file-integrity verification.

Tokenisation

The ALULogoToken contract extends ERC-20 and creates 1,000,000 ALUT tokens. The full supply is minted to the owner at deployment. distributeShares() allows only the owner to transfer shares to other wallets, and ownershipPercentage() computes a wallet's percentage of the fixed total supply. ERC-721 is used for the single unique logo, while ERC-20 is used for interchangeable ownership shares.

Cryptocurrency Exchange Exploration

Coinbase was used as the reference exchange platform. Typical onboarding requests include legal name, date of birth, address, phone number, email, and a government-issued ID. Exchanges require this to meet KYC and AML obligations under laws such as the Bank Secrecy Act and customer-identification rules. The platform separates dashboard, wallet, and trading areas. Bitcoin, Ethereum, and Solana were used as sample listed assets. Bank transfers are generally cheaper, while debit-card purchases are generally faster. A market order was chosen for the purchase simulation because it prioritises execution. Coinbase's public BTC-USD spot endpoint returned 59,811.805 USD per BTC on June 24, 2026, so 100 USD would equal about 0.001672 BTC before trading fees. A transaction hash is the unique blockchain identifier for a transaction and is used to verify settlement independently.

Security Features

Two-factor authentication adds a second login or action-verification step beyond the password, while withdrawal protections such as confirmation prompts and address controls reduce the risk of unauthorised transfers. These measures are especially important because cryptocurrency transfers are difficult or impossible to reverse once confirmed.

Test Results

The project includes eight passing automated tests covering successful asset registration, duplicate-hash rejection, correct and incorrect integrity checks, metadata retrieval, full ERC-20 minting, share distribution, and ownership-percentage calculation.

Conclusion

The project shows how blockchain can provide authenticity, timestamping, and tokenised ownership for a university brand asset. A natural next step would be the dApp interface mentioned in the assignment so users can upload a logo file, compute its hash, verify its authenticity, and distribute ownership shares through a browser interface.

GitHub Link

<https://github.com/tejirijesse/are-you-a-blockchain-developer>

Evidence

The following figures document the generated hash, a passing local test run, and the local Hardhat account addresses.

Figure 1. SHA-256 hash generated for the ALU logo

```
SHA-256: 727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
bytes32: 0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
Source: https://www.alueducation.com/wp-content/uploads/2016/02/alu_logo_original.png
```

Figure 2. Local Hardhat test run

```
> are-you-a-blockchain-developer@1.0.0 test
> hardhat test
```

ALU blockchain project

- ▣ registers the ALU logo successfully and returns token ID 1 (274ms)
- ▣ rejects registering the same hash twice
- ▣ returns true when verifyLogoIntegrity receives the correct hash
- ▣ returns false when verifyLogoIntegrity receives an incorrect hash
- ▣ returns the correct asset metadata for a registered token
- ▣ mints the full ALUT supply to the owner on deployment
- ▣ distributeShares transfers the requested amount to the recipient
- ▣ ownershipPercentage returns the correct whole-number percentage

8 passing (337 ms)

Figure 3. Hardhat local blockchain addresses

Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts

=====

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Account #2: 0x3C44CdDdB6a900fa2b685dd299e03d12FA4293BC (10000 ETH)
Account #3: 0x90F79b6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Account #6: 0x976EA74026E726554dB857fA54763abd0C3a0aa9 (10000 ETH)
Account #7: 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (10000 ETH)
Account #8: 0x23618e81E3f5cdF7f54C3d65f7Fbc0aBf5B21E8f (10000 ETH)
Account #9: 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000 ETH)